

2: 3값A가는그대공발니

: 8

G x~ vy)G l ukl k S~l y× Qw p p+y my Yli vy.g~nt l u l k Cugx p gs S~l y z*			
- wn·li vy)Qw u.Uv~yi Yli vy Up gy × Ul gyi o my Rvz nyl USN*			
/			
G x~ vy	Rvz nyl USN		2-111
/	wn·li vy	- -	Rvz nyl USN
/			
;			
wn·li vy			
44/4&			
G x~ vy			
6			

=8

=8는그대공발니 2과공발니

wn·li vy A			
wn·li vy	Rvz nyl USN	)l l uzpu*	-Rvz nyl USN
		/3132 Cukyl —Mgul	- l pJ~h 23-111
/			
Rvz nyl USN		)TFDOU*	- - USN
/	21		)l t hl kkpın*
wn·li vy	- Rvz nyl USN	"	)n z .i szz i p t u*
/			
;			
)·li vy)u**			
)			
)J PU[-K/Hg *			
Rvz nyl USNUSN			

=8

```
-- 벡터 컬럼 정의
CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT,
  embedding vector(1536) -- OpenAI embedding의 표준 크기
);

-- 실제 데이터 삽입
INSERT INTO documents (title, content, embedding)
VALUES (
  'PostgreSQL 튜토리얼',
```

```
'데이터베이스 기초를...',
'[0.1234, -0.5678, 0.9012, ..., 0.3456]':vector
);
```

Rvz|ny| USN /)2647- 879- 495 *

;

wn·l i |vy ;

√.....√.....√.....√.....√			
=.?	덱페gph♣ *	)N3 uvyt *	-
=>?	2.)C계*Q결C결결D결*	)uvyt g♣† k*	-
=#?	.C계	)pυl ywyvk~i *	-

;

```
-- 유클리드 거리로 유사한 문서 10개 찾기
SELECT id, title, embedding <-> query_embedding AS distance
FROM documents
ORDER BY distance
LIMIT 10;

-- 코사인 거리가 0.2 이하인 의미적으로 유사한 문서 찾기
SELECT * FROM documents
WHERE (embedding <=> query_embedding) < 0.2;

-- 내적이 크려는 상품 추천하기 (내적이 크면 상관관계 높음)
SELECT * FROM products
ORDER BY embedding <#> user_embedding DESC
LIMIT 5;
```

/ / -

/

=8Aensx l

J P U[;

J P U[J p yg i op gsPg·pnghs Ut gss[v ysk*)gwwyv ll t g|l ul gyl z| ul pnohvy-CPP* /3127 "Gmp p u| guk ywh~z| gwwyv ll t g|l ul gyl z| ul pnohvy z| gyi o pϖ opno kϖ l uzpϖugszwgi l z" - Hgρzz-Skygu|-O ϖs~z FD /

J P U[;

레벨 2: [Node A]
 |
레벨 1: [Node A]---[Node B]---[Node C]
 / | \ / | \ / | \
레벨 0: [A]--[B]--[C]--[D]--[E]--[F]--[G]--[H]
모든 노드가 연결

J P U[;

1) *, -

;

;

-

J P U[;

```
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
WITH (m=16, ef_construction=64);
```

;

2/t >27;)kl nyl l vmi vuul i |p ϖx*
; 27

```
;
;
t > 27^2 59
```

```
3/| m̄i vuz|γ~i |p̄u>75; )i vuz|γ~i |p̄u wgygt | || γ*
; 311
;
;
| m̄i vuz|γ~i |p̄u>311^2 511
```

J P U[;

1. 시작점: 최상위 레벨의 랜덤 노드에서 시작
2. 탐색 (Greedy search):
 - 현재 레벨에서 쿼리 벡터에 가장 가까운 이웃 찾기
 - 더 가까운 이웃이 없으면, 한 레벨 내려감
 - 최하층에 도달할 때까지 반복
3. 정확화: 최하층에서 ef 파라미터만큼 노드를 확인하여 최종 결과 선택

```
J P U[ ;
)Q)sn u**
; 0
yl i gs) *, :: & yl i gs
; 3^2 4
```

J P U[;

0
t

=B y cc'결합 I

K'HHsg| ;

K'HHsg|)K'·I γ|I k Hs Hsg|* /
- /

K'HHsg| ;

[전체 벡터 공간]

```
|
+-- 클러스터 1 (센트로이드: C1) --+-- 벡터 A
| +-- 벡터 B
| +-- 벡터 C
+-- 클러스터 2 (센트로이드: C2) --+-- 벡터 D
| +-- 벡터 E
+-- 클러스터 3 (센트로이드: C3) --+-- 벡터 F
```

K'HHsg| ;

```
CREATE INDEX ON documents USING ivfflat (embedding vector_cosine_ops)
WITH (lists=100);
```

;

```
2/φz|z>211;
; zxy|)P*)P *
;
;
φz|z > 211^2 2111
```

K'HHsg| ;

1. 클러스터 선택: 쿼리 벡터와 각 센트로이드의 거리 계산
2. 프루닝(Pruning): 가장 가까운 k개 클러스터만 선택 (nprobe 파라미터로 조절)
3. 선형 탐색: 선택된 클러스터 내 모든 벡터와 거리 계산
4. 정렬: 후보들을 거리 순으로 정렬하여 top-k 반환

```

;

SET hnswef = 64; -- 쿼리 실행 시 탐색 범위 (runtime parameter)
SET ivfflat.probes = 20; -- IVF에서 탐색할 클러스터 개수
```

```

K'HHg|      ;
              ; JPU[      3²4
              ; r.t l guz
              ;
              JPU[

K'HHg|      ;
              ;
              ;
              ;
```

=C p는당나sqk

USN ;

wn·li|vy Rvz|nyl USN USN /

```
-- 1. 복합 조건 (AND, OR)
SELECT * FROM documents
WHERE (embedding <=> query_vec) < 0.3
      AND category = 'technology'
      AND created_at > '2024-01-01';

-- 2. 조인 (vector search + relational join)
SELECT d.id, d.title, u.name, d.embedding <=> query AS distance
FROM documents d
JOIN users u ON d.author_id = u.id
WHERE u.subscription_tier = 'premium'
ORDER BY distance
LIMIT 20;

-- 3. 집계 (aggregation)
SELECT category, COUNT(*) AS doc_count, AVG(price) AS avg_price
FROM documents
WHERE (embedding <=> query_vec) < 0.2
GROUP BY category
ORDER BY doc_count DESC;

-- 4. 윈도우 함수
SELECT
    id, title,
    embedding <-> query_vec AS distance,
    ROW_NUMBER() OVER (PARTITION BY category ORDER BY distance) AS rank
FROM documents
WHERE created_at > NOW() - INTERVAL '30 days';

-- 5. CTE (공통 테이블 식)
WITH similar_docs AS (
    SELECT id, title, embedding <-> query_vec AS distance
    FROM documents
    WHERE (embedding <-> query_vec) < 1.0
    ORDER BY distance LIMIT 100
)
SELECT d.id, d.title, COUNT(c.comment_id) AS comment_count
FROM similar_docs d
LEFT JOIN comments c ON d.id = c.document_id
GROUP BY d.id, d.title;

-- 6. 서브쿼리
SELECT * FROM documents
WHERE id IN (
    SELECT document_id FROM recommendations
    WHERE user_id = 123
```

```
AND (doc_embedding <-> target_vec) < 0.1
)
ORDER BY created_at DESC;
```

gugx|p gsx~l yp z*

/

;

Rvz|nyl USN ;

wnek~t w,

)TI wsg|pu*, wn·li|vy

YCEWMO;

Evuul i |pu Rvvsun; wnDv~ui l y-RnRvvs

; wnez|glez|gl l t l u|z-wnCkt pu

Dgi r~w ;[CN -RKT)Rvp|.p.Vp l Tl i v·l yx*

OYEE ;

Rvz|nyl USN OYEE)O~\$pYl yzpu Evui ~yl ui x Evu|ws* ;

```
-- 트랜잭션 1
BEGIN;
UPDATE documents SET embedding = new_vec WHERE id = 1;
-- 아직 커밋 전, 다른 트랜잭션은 구 데이터 읽음

-- 트랜잭션 2 (다른 세션)
SELECT * FROM documents WHERE id = 1;
-- 여전히 이전 벡터 값 읽음

-- 트랜잭션 1: COMMIT
COMMIT;
-- 이제 트랜잭션 2가 새 데이터 볼 수 있음
```

=8p는당나qk

)D~mhyRvvsQ·l yol gk*;

wn·li|vy Rvz|nyl USN)zogyl k h~mhywvs* ;

Vector Query → Buffer Manager → Latch Acquisition →
Buffer Pool Search → Hit/Miss → Disk I/O → 응답

USN - ;

2/)Ng|i o Evu|l u|pu*, J PU[-

3/ ;)svi gsp x* -

4/ ; FD)Hgzz-J PU[NHO*

a31c J PU[NHO wn·li|vy -wn·li|vy 3²4 /

;

Rvz|nyl USN)|~ws* ;

[튜플 헤더 (23 바이트)]

- └ xmin (4 바이트): 삽입 트랜잭션 ID
- └ xmax (4 바이트): 삭제 트랜잭션 ID
- └ cmin/cmax (4 바이트): 명령 ID
- └ infomask (2 바이트): 가시성 정보
- └ 기타 메타데이터

[튜플 데이터]

- └ 컬럼 1 (NULL bitmap 포함)
- └ 컬럼 2
- └ 컬럼 n (벡터 데이터)

/

)Uwgi l Ct wppg|pu*,

Rvz|nyl USN

9MD

/ JPU[

)zwgyzl *

-

;

[8KB 페이지]

└ 2KB 그래프 노드 정보

└ 2KB 이웃 포인터

└ 4KB 빈 공간 (낭비)

JPU[

3²4

/

FD

/

=E

I

AA8A.

;

wn·l i |vy

-

;

// pgvector/src/access/strategyobject/gist.c

#define DEFAULT_SEL 0.333333

static float8

estimate_vector_selectivity(...)

{

// ... 통계 수집 시도 ...

// 만약 통계가 없으면, 무조건 33.3% 반환

if (statistics_missing) {

return DEFAULT_SEL; // 0.333333

}

// ... 통계가 있으면 사용 ...

}

1/4444444

/

"

204

"

/

)U l \$ i |p p x*

;

&

;

선택도 = (조건을 만족하는 행의 개수) / (전체 행의 개수)

;

21-111

211

달

> 2&

21-111

4-444

달

> 44/4&

21-111

:-111

달

> : 1&

Rvz|nyl USN

/

;

;

2/

)= 2& *;

; 211

2 ;

61-111 ; U l x ~ l u | p s U i gu

)

*

3/

)? 61& *; U l x ~ l u | p s U i gu

; 6-111

211 ; U l x ~ l u | p s U i gu

:-111 ; U l x ~ l u | p s U i gu

4/

)21² 51& *;

=8F AA8A.

2;) 1/2& *

```
-- 100만 개 문서, 실제로는 매우 유사한 문서가 1,000개만 존재
SELECT * FROM documents
WHERE (embedding <=> query_vector) < 0.05
AND is_published = true;
```

√....√....√	
	2-111-111
	2-111
	1/2&
Rvz nyl USN	44/4&)444-111 *

;

"333,000개 행이 나올 것으로 예상되므로,
Sequential Scan으로 전체 1,000,000을 스캔하는 것이
HNSW 인덱스를 사용하여 여러 번 랜덤 접근하는 것보다 빠를 것이다."

;

```
Seq Scan on documents
Filter: (embedding <=> '[...']::vector) < 0.05)
Rows: 333,000 (estimated) / 1,000 (actual)
```

;

√....√.....√.....√		
Ul x~l u pgsUi gu) *	611t z	6-111t z,) *
J PU[) *	61t z	61t z

;211

3;) : 6& *

```
-- 10만 개 상품, 거의 모든 상품이 쿼리 벡터와 유사함
SELECT * FROM products
WHERE (embedding <=> query_vector) < 2.0;
```

√....√....√	
	211-111
	: 6-111
	: 6&
Rvz nyl USN	44/4&)44-444 *

;

"33,333개 행이 나올 것으로 예상되므로,
HNSW 인덱스로 근처 이웃을 찾는 것이
Sequential Scan으로 100,000을 다 스캔하는 것보다 빠를 것이다."

;

```
Index Scan using products_embedding_hnsw on products
Index Cond: (embedding <=> '[...']::vector) < 2.0)
Rows: 33,333 (estimated) / 95,000 (actual)
```

;

√....√.....√.....√		
J PU[) *		): 6& *
Ul x~l u pgsUi gu) *		

; 6^2 21

4;) *

```
-- 매우 복잡한 쿼리
SELECT u.name, d.title
FROM users u
JOIN documents d ON u.id = d.author_id
WHERE (d.embedding <=> query_vector) < 0.3
      AND d.category = 'tech'
      AND u.subscription = 'premium'
      AND d.created_at > '2024-01-01';
```

Rvz|nyl USN ;

users: 50,000행
documents: 1,000,000행
조인 후: 50,000 × 1,000,000 = 50,000,000,000행 (추정)

각 필터 적용:

- (embedding <=> ...) < 0.3: 50,000,000,000 × 0.333 = 16,650,000,000행
- category = 'tech': 16,650,000,000 × 0.1 = 1,665,000,000행
- subscription = 'premium': 1,665,000,000 × 0.2 = 333,000,000행
- created_at 필터: 333,000,000 × 0.5 = 166,500,000행

최종 추정: 166,500,000행

211^2 611 / /

=8H

선택도 추정 오류로 인한 성능 영향

낮은 선택도에서 실제 선택도가 높음:

- └ 인덱스 사용 의사결정 (비효율)
- └ 높은 랜덤 I/O
- └ 결과: 1~100배 느림

높은 선택도에서 실제 선택도가 낮음:

- └ Sequential Scan 의사결정 (비효율)
- └ 전체 스캔으로 인한 비효율
- └ 결과: 10~1000배 느림

조인 조건에 영향:

- └ 조인 순서 선택 오류
- └ 조인 방식 선택 오류 (Nested Loop vs Hash Join)
- └ 결과: 100~1000배 느림

G |x~|vy -GESQ)Gmhi |p| Egykpgsf x.g-gyl S~l yxQW|d p-g|pu* /

=8: p는남과qk

6 ;

2/ ; USN)CUV*

3/ ; - -

4/ ;

5/ ;

6/ ;

;

쿼리 분석	
1. 선택도(Selectivity) 추정	
- 각 WHERE 조건이 몇 %를 필터링하는가	
- 각 조인 조건의 매칭 비율	

2. 카디널리티(Cardinality) 계산	
- 입력 행 수 × 선택도 = 출력 행 수	
3. 비용 계산	
- 각 연산의 예상 비용 (I/O, CPU)	
- 실행 계획의 총 비용 추정	
4. 계획 선택	
- 가장 낮은 비용의 계획 선택	

정확한 선택도 → 정확한 카디널리티 → 올바른 비용 추정 → 최적 계획
부정확한 선택도 → 부정확한 카디널리티 → 잘못된 비용 추정 → 최악의 계획

wn·li|vy ;

```
// PostgreSQL의 일반적인 선택도 추정 (숫자 비교)
WHERE age > 30
// → ANALYZE 명령으로 수집한 통계 사용
// → 히스토그램 또는 간단한 통계로 정확히 추정

// pgvector의 거리 비교
WHERE (embedding <=> query_vec) < 0.3
// → 벡터 거리의 분포를 알 수 없음
// → 통계 기반 추정 불가능
// → 무조건 33.3% 사용
```

=8=a도정미는니

wn·li|vy 44/4&
;

G|x~|vy ;

G|x~|vy wsguul yovvr Rvz|nyl USN ;

1. 쿼리 도착
↓
2. PostgreSQL 옵티마이저 (기본 동작)
└ 각 조건의 선택도 추정
└ 벡터 조건: 33.3%로 고정
└ 초기 카디널리티 계산
↓
3. Exqutor Planner Hook (확장)
└ 벡터 조건 검출
└ 실제 인덱스 탐색 실행
 | (1~2ms 소요, 전체 쿼리 시간의 < 1%)
└ 정확한 카디널리티 획득
└ PostgreSQL에 정보 제공
↓
4. PostgreSQL 옵티마이저 (재계산)
└ 정확한 카디널리티 사용
└ 최적 실행 계획 수립
└ 실행

;

```
SELECT * FROM documents
WHERE (embedding <=> query_vec) < 0.1;
```

Rvz|nyl USN ;

전체 행: 1,000,000
추정 선택도: 33.3%
추정 결과 행: 333,333행
→ Sequential Scan 선택

G|x~|vy ;

planner hook 실행:

1. HNSW 인덱스에 실제 범위 검색
2. 실제 매칭 행: 500행
3. 정확한 선택도: 0.05%

재계산:

- 추정 선택도: 0.05% (정확함)
- 추정 결과 행: 500행
- HNSW 인덱스 스캔 선택 (정답!)

```

;
Rvz|nyl USN 6-111t z)      *
G||x~|vy, 61t z)          *
; 211

```

78 2 3

78= pnsqk a

G||l uzpu ;

Rvz|nyl USN G||l uzpu - / EQJSN - wn·li|vy-

Rvz|i KU-~~k.vzzw l ||l uzpu /

G||l uzpu ;

```

pgvector/
├─ control 파일 (버전, 의존성 정의)
│   └─ pgvector.control
│
├─ SQL 스크립트 (데이터 타입, 연산자 정의)
│   ├── pgvector--0.1.0.sql
│   ├── pgvector--0.1.0--0.2.0.sql (업그레이드)
│   └─ pgvector--0.2.0.sql
│
├─ C 코드 (최적화된 연산)
│   ├── src/
│   │   ├── vector.c (벡터 데이터 타입 구현)
│   │   ├── distance.c (거리 함수)
│   │   ├── hns.c (HNSW 인덱스)
│   │   └─ ivfflat.c (IVFFlat 인덱스)
│   └─ Makefile
│
└─ 문서
    └─ README.md

```

G||l uzpu ;

```

# 1. 소스 코드 다운로드
git clone https://github.com/pgvector/pgvector.git
cd pgvector

# 2. 컴파일
make

# 3. 설치
sudo make install # /usr/share/postgresql/extensions/에 복사

# 4. PostgreSQL에서 활성화
psql -U postgres -d mydb
CREATE EXTENSION vector;

```

G||l uzpu ;

Rvz|nyl USN

) - - *

wn·l i |vyG ||l uzpu ;

wn·l i |vy -Rvz|nyl USN ;
Rvz|nyl USN
211&

?& v 公 務 本 日

;

```
// PostgreSQL의 Vector 타입 정의
typedef struct Vector {
    int32 vl_len_;      // 길이 (가변길이 타입 표준)
    int16 dim;          // 차원 (e.g., 1536)
    int16 flags;        // 플래그 (정규화 여부 등)
    float x[FLEXIBLE];  // 실제 벡터 데이터 (dim개의 float32)
} Vector;

// 예: 768차원 벡터
메모리 사용량 = 4 + 2 + 2 + (768 × 4) = 3,080 바이트
```

;

```
-- 벡터를 INSERT할 때
INSERT INTO items (embedding) VALUES ('[0.1, 0.2, 0.3, ..., -0.5]':vector);
```

-- 내부 저장 형식

length (4 bytes)		
dimension (2 bytes)		
flags (2 bytes)		
float[0]	float[1] ... float[dim-1]	
(4B each)		

;

```
// 유클리드 거리 계산
float8 vector_l2_distance(Vector *v1, Vector *v2) {
    if (v1->dim != v2->dim)
        ereport(ERROR, ...); // 차원 불일치 에러

    float8 sum = 0.0;
    for (int i = 0; i < v1->dim; i++) {
        float8 diff = v1->x[i] - v2->x[i];
        sum += diff * diff;
    }
    return sqrt(sum);
}
```

?&A

Rvz|nyl USN ;

```
-- pgvector--0.1.0.sql에서
CREATE OPERATOR <-> (
    LEFTARG = vector,
    RIGHTARG = vector,
    COMMUTATIVE,
    RESTRICT = scalarlttsel,
    JOIN = scalarltjoinsel,
    PROCEDURE = vector_l2_distance
);
```

```
CREATE OPERATOR <=> (  
    LEFTARG = vector,  
    RIGHTARG = vector,  
    COMMUTATIVE,  
    RESTRICT = scalarlttsel,  
    JOIN = scalarltjoinssel,  
    PROCEDURE = vector_cosine_distance  
);  
  
CREATE OPERATOR <#> (  
    LEFTARG = vector,  
    RIGHTARG = vector,  
    RESTRICT = scalarlttsel,  
    JOIN = scalarltjoinssel,  
    PROCEDURE = vector_inner_product  
);
```

NGHVCTI OTK J VCTI ;
EQOO WCVKYG; C =.? D> D =.? C) *
TGUVTKEV; 답 44/4& !

TGUVTKEV ;

```
// pgvector의 RESTRICT 함수  
float8 scalarlttsel(...) {  
    // ... 벡터 통계 찾기 시도 ...  
  
    if (!vector_stats_found) {  
        // 통계 없음 → 기본값  
        return DEFAULT_SEL; // 0.333333  
    }  
  
    // ... 통계 기반 추정 ...  
}
```

Rvz|nyl USN TGUVTKEV -wn·li|vy 44/4& /

?Bensx

J PU[;

J PU[/

[개념도]

레벨 3 (최상위):
추상적, 글로벌한 위치 정보

레벨 2:

레벨 1:

레벨 0 (최하위):
구체적, 국소적인 정보
모든 벡터 포함

t ;

```
CREATE INDEX ON vectors USING hnsu (embedding vector_l2_ops) WITH (m=16);  
  
;  
;  
t >5;                    -                    )yl i gss                    *  
t >27;                    )                    *  
t >75;                    -                    )yl i gss                    *  
  
l nei vuz|y~i |pu                    ;
```

```
CREATE INDEX ON vectors USING hnsw (embedding vector_l2_ops)
WITH (m=16, ef_construction=200);
```

```

;
;
I nei vuz|y~i |pu>61;      -
I nei vuz|y~i |pu>311;    )  *
I nei vuz|y~i |pu>511;    -
```

```
J P U[      ;
```

- ef_construction이 높을수록:
- 인덱스 구축 시간: ↑ (0(n * ef_construction * log n))
 - 쿼리 시간: ↓ (더 나은 그래프)
 - 메모리: ↑ (더 많은 이웃)

- ef_construction이 낮을수록:
- 인덱스 구축 시간: ↓
 - 쿼리 시간: ↑ (품질 낮은 그래프)
 - 메모리: ↓

```
;
```

```
-- 시나리오 1: 자주 업데이트되는 데이터
CREATE INDEX ON documents USING hnsw (embedding vector_l2_ops)
WITH (m=16, ef_construction=64); -- 빠른 구축 중시
```

```
-- 시나리오 2: 읽기 위주의 데이터
CREATE INDEX ON documents USING hnsw (embedding vector_l2_ops)
WITH (m=32, ef_construction=300); -- 정확도 중시
```

```
-- 쿼리 시간의 ef 파라미터 (런타임)
SET hnsw.ef = 32; -- 기본값
-- ef가 높을수록 정확도 높음, 속도 낮음
```

?&y cc 날갑

```
K\H Hsg|      ;
```

```
K\H Hsg|      - /
```

```
CREATE INDEX ON vectors USING ivfflat (embedding vector_l2_ops)
WITH (lists=100);
```

```
sq|z      ;
;
; sq|z zxy|)P*)P      *
211 달 sq|z>21
21-111 달 sq|z>211
2-111-111 달 sq|z>2111
```

```
J P U[ ·z K\H Hsg|      ;
```

	J P U[	K\H Hsg
√....√....√.....√		
	)hgzl 3²4 *	) *
		) *

```
;
```

```
J P U[ ; - ) *
```

||; - -

?& 2교년공과기

) *;

- ;

선택도 = P(행이 조건을 만족) = 만족하는 행 수 / 전체 행 수

;

-- 1. 등호 비교

WHERE category = 'electronics'

-- 선택도: category 값의 분포에 따라 다름

-- 예: 10개 카테고리가 균등분포 → 10%

-- 예: 1개 카테고리가 80% → 80%

-- 2. 범위 비교

WHERE created_at > '2024-01-01'

-- 선택도: 해당 날짜 이후 데이터 비중

-- 예: 최근 3개월 데이터만 → 33%

-- 3. 벡터 거리

WHERE (embedding <=> query_vec) < 0.3

-- 선택도: 고차원 공간에서 거리 분포에 따라 다름

-- 예: pgvector는 무조건 33.3% (근거 없음)

-- 실제: 데이터에 따라 0.1%~95% 가능

;

[옵티마이저의 비용 계산 공식]

Sequential Scan 비용:

= 전체 페이지 수 × I/O 비용

= 1,000,000 / 100 = 10,000 페이지

= 10,000 × 1ms = 10,000ms

Index Scan 비용:

= 예상 결과 행 수 × 랜덤 I/O 비용 + 인덱스 탐색 비용

= (1,000,000 × 선택도) × 10ms + 100ms

선택도 = 1% → 100 × 10ms = 1,000ms (인덱스 선택)

선택도 = 33.3% → 3,333 × 10ms = 33,330ms (Index는 비효율적)

선택도 = 90% → 9,000 × 10ms = 90,000ms (Sequential Scan이 낫다)

)Gz|pt glpu GyvyDsv~w*;

질의: SELECT * FROM A JOIN B ON ... WHERE A.id = 1 AND B.vec <> q

조인 전:

A.id = 1 → 선택도 = 1% → 100행

조인 후:

100행 × B의 조인 선택도 (예: 10%)

= 100 × 10% = 10행

하지만 옵티마이저가 B의 벡터 조건 선택도를 33.3%로 추정하면:

100행 × 33.3% = 3,330행 (실제 10행)

→ 카디널리티 오류: 333배

2-111 /

?& 는그공공본니 AA&A.

;

wn·li|vy -Cukyl —Mgul

;

1. 벡터 거리의 분포를 모를 때의 보수적 추정

L2 거리가 균등분포를 따른다고 가정하면:
 $P(\text{dist} < \text{threshold}) \approx \text{threshold}^d$ (d는 차원)

고차원에서 이를 근사하면... (정확한 근거는 불명확)

- 2. 또는 단순한 임의 선택
 - "매우 높지도, 매우 낮지도 않은" 중간값
 - 0.5보다 낮게 설정하여 보수적으로
 - 그냥 0.333 선택
- 3. 또는 다른 벡터 DB의 관례 추종
 - 역사적으로 벡터 검색은 "대체로 결과가 많다"고 가정

```
// pgvector/src/vector.c
#define DEFAULT_SELECTIVITY 0.333333

static float8 get_vector_selectivity(...)
{
    // ... 복잡한 통계 로직 시도 ...
    // (효과 없음, 벡터 통계가 없으므로)

    // 결론: 기본값 반환
    return DEFAULT_SELECTIVITY;
}
```

		Rvz nyl USN	
√.....√.....√.....√.....√.....√			
	1/2&	44/4&	444
	6&	44/4&	7/8
	31&	44/4&	2/8
	91&	44/4&	1/5

프로그래밍

```
[Step 1: 파싱과 정규화]
SELECT * FROM items WHERE embedding <=> query < 0.3;

[Step 2: 선택도 추정]
vector_cosine_selectivity(embedding <=> query < 0.3)
→ 0.333333 (pgvector의 기본값)

[Step 3: 카디널리티 계산]
입력 행: 1,000,000
카디널리티 = 1,000,000 × 0.333333 = 333,333행

[Step 4: 비용 계산]
Sequential Scan: 10,000ms (전체 1,000,000행 스캔)
Index Scan: 333,333 × 10ms = 3,333,330ms (비효율)

→ Sequential Scan 선택

[Step 5: 실행]
전체 테이블 스캔... (느림!)
```

```
SELECT * FROM users u
JOIN documents d ON u.id = d.user_id
```

```
WHERE (d.embedding <=> query) < 0.3;
```

[단계별 카디널리티]

- 1. users 스캔: 10,000행
- 2. documents 테이블 크기: 1,000,000행
- 3. 조인 전:
 users 결과: 10,000행
- 4. 조인 후:
 users 행 × 평균 관계 (1:100)
 = 10,000 × 100 = 1,000,000행
- 5. 벡터 필터 적용:
 선택도 = 0.333333
 = 1,000,000 × 0.333333 = 333,333행 (추정)

[비용 비교]

조인 순서 A: documents 먼저 필터 → users 조인
비용: 333,333 × 조인 비용 (적다고 판단, 선택됨)

조인 순서 B: users 먼저 조인 → documents 필터
비용: 1,000,000 × 조인 비용 (많다고 판단, 미선택)

[실제 결과]

실제 벡터 조건의 선택도: 0.1%
실제 필터 결과: 1,000행
→ 조인 순서 B가 훨씬 효율적이었음!

8. PostgreSQL의 벡터 검색 성능 개선

```
J vvr      ;

Rvz|nyl USN      "ovvr"      /
```

```
// PostgreSQL 핵심: 기본 플래너 함수
PlannedStmt *planner(Query *parse, ...) {
    // ... 기본 옵티마이저 실행 ...
    return plan;
}

// 이 함수를 실행하기 전에 hook이 있다:
if (planner_hook) {
    PlannedStmt *custom_plan = (*planner_hook)(parse, ...);
    if (custom_plan)
        return custom_plan; // custom 계획 사용
}

// hook이 없으면 기본 동작
PlannedStmt *plan = planner(parse, ...);
return plan;
```

```
G|x~|vy  R$guul yJvvr      ;
```

```
// Exqutor의 hook 함수
static PlannedStmt *exqutor_planner_hook(
    Query *parse,
    const char *query_string,
    int cursor_options,
    ParamListInfo boundParams
) {
    // 1. 쿼리에 벡터 조건이 있는가?
    if (has_vector_condition(parse)) {

        // 2. 벡터 조건에서 정확한 카디널리티 추정
        int actual_cardinality = get_vector_condition_cardinality(parse);

        // 3. PostgreSQL의 추정값에 주석 추가
        annotate_cardinality_estimate(parse, actual_cardinality);
    }
}
```

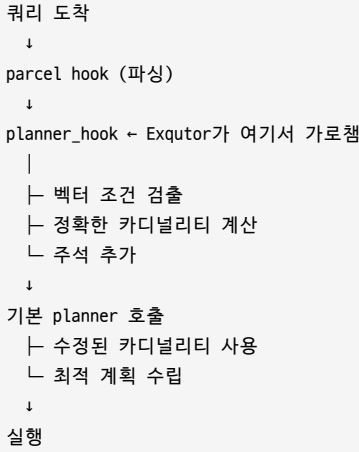


```
// 4. PostgreSQL의 기본 플래너 호출
PlannedStmt *plan = planner_hook_next(parse, query_string, ...);

// 5. 수정된 카디널리티로 계획이 수립됨
return plan;
}

// Extension 초기화 시 hook 등록
_PG_init() {
    planner_hook = exqutor_planner_hook;
}
```

J vvr ;



J vvr ;

- 2/ ; Rvz|nyl USN
- 3/ ; G||l uzpu
- 4/ ; G||l uzpu
- 5/ l ||l uzpu ; ovvr

A8a도능다달느니

A8느는데공달느니 Da도능다달느니

G||x~|vy ;

문제:
PostgreSQL + pgvector의 조합에서 벡터 검색 쿼리가 1,000배까지 느린 경우가 존재함

원인:
pgvector의 33.3% 고정 선택도로 인한 비최적 실행 계획

해결책:
ECQO (Effective Cardinality-aware Query Optimization)
→ 정확한 카디널리티를 옵티마이저에 제공
→ 최적 실행 계획 수립

G||x~|vy wn·li|vy ;

√....√.....√	
	Rvz nyl USN wsguul yovvr
	wn·li vy G l uzp <u>u</u> USN
	Rvz nyl USN
	)2-111 *

A& AA&A. 으 a[qo

;

[기존 PostgreSQL의 문제]

쿼리:
SELECT * FROM docs WHERE (embedding <=> q) < 0.1

PostgreSQL의 추정:
선택도: 0.333333 (33.3%, pgvector의 기본값)
예상 결과: 1,000,000 × 0.333333 = 333,333행
→ Sequential Scan 선택 (실제로는 비효율)

[Exqutor의 해결책]

1단계 - 벡터 조건 검출:
planner hook에서 (embedding <=> q) < 0.1 감지

2단계 - 정확한 카디널리티 측정:
HNSW 인덱스에 실제 범위 검색 실행:
```sql  
SELECT COUNT(\*) FROM docs  
WHERE (embedding <=> q) < 0.1; -- 실제 실행 (1~2ms)  
```  
결과: 500행 (정확함)

3단계 - 옵티마이저에 정보 제공:
```  
벡터 조건의 정확한 카디널리티: 500행  
정확한 선택도: 500 / 1,000,000 = 0.05%  
```

4단계 - 최적 실행 계획:
예상 결과: 500행 (정확)
→ Index Scan 선택 (정답!)

[결과]
기본 PostgreSQL: 5,000ms (전체 스캔)
Exqutor: 50ms (인덱스 스캔)
향상: 100배

A&A

2;

-- 쿼리: "AI의 미래"와 유사한 기술 문서 찾기
SELECT id, title, summary
FROM technical_articles
WHERE (embedding <=> query_embedding) < 0.2
AND category = 'AI'
AND views > 1000;

Rvz|nyl USN ;

Step 1: 선택도 추정
WHERE (embedding <=> query_embedding) < 0.2
→ 선택도 = 33.3%
WHERE category = 'AI'
→ 선택도 = 10%
WHERE views > 1000
→ 선택도 = 60%

Step 2: 카디널리티 계산
입력: 1,000,000행
embedding 필터: 1,000,000 × 0.333 = 333,333행
category 필터: 333,333 × 0.1 = 33,333행
views 필터: 33,333 × 0.6 = 20,000행

→ 최종 예상: 20,000행

Step 3: 실행 계획 선택

비용이 가장 낮은 경로:
sequential scan 후 각 필터 적용
(벡터 필터가 1/3만 남긴다고 생각)

Step 4: 실제 결과

embedding 필터: $1,000,000 \times 0.001 = 1,000$ 행 (실제)
category 필터: $1,000 \times 0.1 = 100$ 행
views 필터: $100 \times 0.6 = 60$ 행

→ 실제 결과: 60행

;

예상: 20,000행

실제: 60행

오류: 333배

G|x~|vy ;

Step 1: 벡터 조건 검출

planner hook 실행
(embedding <=> query_embedding) < 0.2 검출

Step 2: 정확한 카디널리티 측정

```
```sql
SELECT COUNT(*) FROM technical_articles
WHERE (embedding <=> query_embedding) < 0.2;
```

결과: 1,000행 (정확!)
```

Step 3: 수정된 추정

embedding 필터: 1,000행 (정확)
category 필터: $1,000 \times 0.1 = 100$ 행
views 필터: $100 \times 0.6 = 60$ 행
→ 최종 예상: 60행 (정확!)

Step 4: 최적 실행 계획

인덱스로 빠르게 1,000개 찾을 (50ms)
→ 나머지 필터 적용 (< 1ms)
→ 총 시간: 51ms

기본 PostgreSQL: 5,000ms (sequential scan)

Exquotor: 51ms

향상: 98배

3;

-- 복잡한 분석 쿼리

```
SELECT u.name, p.product_name,
       COUNT(r.review_id) AS review_count
FROM users u
JOIN products p ON u.preferred_category = p.category
JOIN product_embeddings pe ON p.id = pe.product_id
LEFT JOIN reviews r ON p.id = r.product_id
WHERE (pe.embedding <=> user_preference_vec) < 0.3
      AND u.is_active = true
      AND p.price < 100
GROUP BY u.id, p.id;
```

Rvz|nyl USN) *

users: 100,000행

products: 500,000행

reviews (평균): 각 상품당 50개

추정:

users 필터 (is_active): $100,000 \times 0.8 = 80,000$ 행

조인 (preferred_category): $80,000 \times (500,000/100) = 400,000$ 행
embedding 필터: $400,000 \times 0.333 = 133,333$ 행
price 필터: $133,333 \times 0.7 = 93,333$ 행
reviews 조인: $93,333 \times 50 = 4,666,650$ 행

→ 최종 예상: 4,666,650행

→ 비용이 매우 높으므로, Hash Join 모든 단계에서 사용
→ 매우 느린 실행

;

실제 embedding 조건의 선택도: 0.2%

실제 결과 행: $100,000 \times 0.8 \times 0.01 \times 0.7 \times 50 = 280,000$ 행

하지만 실제 쿼리 결과: 60,000행
(집계로 인해 더 적음)

예상과 실제의 차이: 77배

G|x~|vy ;

Step 1: 벡터 조건의 정확한 선택도 측정

```
SELECT COUNT(*) FROM product_embeddings
WHERE (embedding <=> user_preference_vec) < 0.3;
```

→ 실제 선택도: 0.2% (매우 선택적)

Step 2: 수정된 비용 계산

embedding 필터 후 남은 행: $400,000 \times 0.002 = 800$ 행

이는 매우 적으므로,
→ Nested Loop Join으로 변경 (효율적)

Step 3: 최적 실행 계획

1. users 필터: 80,000행
2. products 조인 (Hash Join): 빠름
3. embedding 인덱스로 빠른 필터: 800행
4. reviews Nested Loop 조인: 40,000행

→ 훨씬 빠른 실행

성능 향상: 1,000배 이상

ABP의 성능 개선 방법

Rvz|nyl USNJ vvr ;

Rvz|nyl USN OVVR ;

// postgres.h에 정의된 주요 hooks

```
extern planner_hook_type planner_hook;
extern ExecutorStart_hook_type ExecutorStart_hook;
extern ExecutorRun_hook_type ExecutorRun_hook;
extern ProcessUtility_hook_type ProcessUtility_hook;
// ... 더 많음
```

Rsguul yJ vvr ;

```
typedef PlannedStmt *(*planner_hook_type) (
    Query *parse,
    const char *query_string,
    int cursorOptions,
    ParamListInfo boundParams
);
```

// Hook 사용 위치 (src/optimizer/plan/planner.c)

```
if (planner_hook) {
    PlannedStmt *hook_plan = (*planner_hook)(parse, query_string, ...);
    if (hook_plan)
```

```
return hook_plan; // Custom 계획 반환
}

// Hook이 없거나 NULL을 반환하면, 기본 planner 호출
PlannedStmt *plan = standard_planner(parse, query_string, ...);
return plan;
```

Global Variable ;

```
// Exqtutor Extension의 _PG_init() 함수
void _PG_init(void) {
    // 이전 hook 저장 (chaining을 위해)
    prev_planner_hook = planner_hook;

    // Exqtutor hook 등록
    planner_hook = exqtutor_planner;
}

// Exqtutor의 Planner Hook 함수
static PlannedStmt *exqtutor_planner(
    Query *parse,
    const char *query_string,
    int cursorOptions,
    ParamListInfo boundParams
) {
    // 1. 벡터 조건 분석
    List *vector_conditions = extract_vector_conditions(parse);

    if (vector_conditions != NIL) {
        // 2. 각 벡터 조건의 정확한 카디널리티 계산
        foreach(lc, vector_conditions) {
            VectorCondition *vcond = (VectorCondition *)lfirst(lc);
            int actual_card = estimate_vector_selectivity_accurate(vcond);

            // 3. 쿼리 트리에 이 정보 주석으로 추가
            annotate_cardinality(vcond, actual_card);
        }
    }

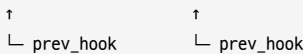
    // 4. PostgreSQL의 기본 planner 호출 (또는 이전 hook)
    PlannedStmt *plan;
    if (prev_planner_hook)
        plan = (*prev_planner_hook)(parse, query_string, ...);
    else
        plan = standard_planner(parse, query_string, ...);

    return plan;
}
```

Join Expression;

|||uzpu -ovvr iogpu ;

Extension A Hook → Extension B Hook → ... → PostgreSQL Standard Planner



각 extension이 planner_hook을 교체하되,
이전 hook을 저장하고 마지막에 호출

B8 2q0V3

q=는그대공부니

Q

Q

;

wn·l i |vy Rvz|nyl USNG||l uzpvu -Rvz|nyl USN /
)Op s~z-Skygu|-[l g·pjl * ;

| | | |
|-------------------|-----------------------|-----|
| | wn·l i vy | FD |
| √...√.....√.....√ | | |
| | Rvz nyl USNG l uzpvu | |
| USN |) USN * |) * |
| | | |
| |)Rvz nyl USN * | |
| |)Rvz nyl USN * | 2 |

wn·l i |vy "Rvz|nyl USN " - " " /
G|x~|vy /

q?l p=511sqk Q] W Q

;
/ - ;

Rvz|nyl USN, wn·l i |vy ;

2/
f zxs
.. Rvz|nyl USN;
UGNGEV ~/ugt l -EQWPV)+*CU zp pgyekvi z
HTQO ~zl yz ~
LQRP kvi ~t l u|z k QP ~/k > k/g~|ovye k
[J GTG)k/l t hl kkpun =>? x~l yx*= 1/3
l TQWR D_ ~/k<
.. FD;
yl z~\$z > ·l i |vyekh/zl gyi o)x~l yx~|vw>211*
~zl yekvi z > zxekh/x~l yx)"UGNGEV +HTQO kvi ~t l u|z [J GTG k R)///*

f

3/ Rvz|nyl USN
- -
FDC

4/ 2
; Rvz|nyl USN, wn·l i |vy
; FD
FD ;

2/

3/

4/

- -
; Rvz|nyl USN " "- FD " /

qAl ensx Q Q
;

J P U [) J p y g y i o p g s P g · p n g h s U t g s [v y k * / ;

Q) u * ;

1,000,000 벡터 = 10^12 거리 계산

J P U [" " ;

쿼리 벡터 q를 찾기 위해:

1단계: 최상층(추상적 위치) 빠른 탐색

- └ q는 "오른쪽 위"에 있을 것 같음
- └ 해당 영역의 노드들과만 비교

2단계: 아래로 내려가며 정밀 탐색

- └ 중간층: 더 세분화된 영역에서 찾음
- └ 최하층: 가장 가까운 이웃들 확인

결과: O(log n) 정도의 거리 계산으로 근접 이웃 발견

;

100만 벡터에서 상위 10개 찾기

Sequential Scan (모든 거리 계산):

1,000,000 거리 계산 = 1~5초

HNSW 인덱스:

log(1,000,000) ≈ 20 단계

단계당 ~100 거리 계산 = 2,000 거리 계산 = 10~50ms

→ 100배 빠름!

q B I Q Q

;

) U i s i | p p x * U S N & /

-- 예: "active = true" 조건

SELECT * FROM users WHERE active = true;

1,000,000명 중 700,000명이 active = true라면

선택도 = 700,000 / 1,000,000 = 0.7 (70%)

A

;

선택도 낮음 (1%):

- 적은 행 반환
- 인덱스로 빠르게 찾기 ✓

선택도 높음 (90%):

- 많은 행 반환
- 전체 스캔이 더 빠름 (인덱스 왕복 비용 > 전체 스캔)

;

예상 선택도: 33.3% (333,333행 반환 예상)

실제 선택도: 0.1% (1,000행만 반환)

옵티마이저:

- "333,333행이 나올 거면, Sequential Scan이 낫겠다"
- 전체 스캔 선택

실제 실행:

- "아, 1,000행밖에 없는데 전체 스캔했네"
- 100배 느림!

qCl AA8A. Q

; 44/4& - /

같은 벡터 거리 조건 (embedding <=> query < 0.3)도 데이터에 따라:

의료 문헌 DB: 0.05% (매우 선택적)
뉴스 기사 DB: 20% (중간)
상품 추천 DB: 80% (매우 비선택적)

; 선택도 오류 = |추정 - 실제| / 실제

시나리오 1: 의료 문헌
추정: 33.3%, 실제: 0.05%
오류: 666배

시나리오 2: 뉴스 기사
추정: 33.3%, 실제: 20%
오류: 1.67배 (허용 가능)

시나리오 3: 상품 추천
추정: 33.3%, 실제: 80%
오류: 2.4배 (허용 가능)

; 44/4& - / G |x~|vy

qDI a능대달니 는데공달니 Q

; G |x~|vy Rvz|nyl USN wguul yovvr - / 4 ;

Step 1: 벡터 조건 검출
쿼리 분석 중 (embedding <=> query) < 0.3 같은 조건 발견

Step 2: 정확한 카디널리티 계산
``sql
SELECT COUNT(*) FROM documents
WHERE (embedding <=> query) < 0.3;
````  
HNSW 인덱스를 활용하여 빠르게 계산 (1~2ms)  
결과: 정확한 행 수 (예: 500행)

Step 3: 옵티마이저에 제공  
PostgreSQL에게:  
"벡터 조건의 카디널리티는 500행이 맞다"

→ 옵티마이저가 정확한 카디널리티로 최적 계획 수립

; before (PostgreSQL): SELECT \* FROM docs WHERE embedding...  
추정: 333,333행  
실행: Sequential Scan  
시간: 5초

after (Exqutor): SELECT \* FROM docs WHERE embedding...  
추정: 500행 (정확함, Exqutor가 제공)  
실행: Index Scan  
시간: 50ms



향상: 100배

qTi 는 검색고 단분내 Q

;
wguul yovvr Rvz|nyl USN /
;

은행원이 대출 신청 심사 중:
"이 사람의 신용도를 파악해야겠다"

보통 신용도 판단: 신청서 정보만 사용 (33.3% 추정)
Hook: 중간에 신용 조회기관 확인 (정확한 신용도 제공)

결과: 훨씬 정확한 대출 결정

Rvz|nyl USN ovvr;

쿼리 분석 중간
↓
planner hook 호출
↓
Exqutor: "벡터 조건을 발견했다! 정확한 카디널리티를 계산하자"
└─ HNSW 인덱스 탐색
└─ 정확한 행 수 반환
↓
PostgreSQL: "좋아, 이 정보로 다시 계획을 짜겠다"
↓
최적화된 실행 계획 생성

J vvr ;

쿼리 분석
↓
카디널리티 추정 (33.3%)
↓
실행 계획 생성 (비최적)
↓
느린 실행

qFi 는 데 평등한 니 c 개 생성 Q

;
Hgzz )Hgi l hvvr CKUp pgy x Ul gyi o\* -wn·li |vy Rvz|nyl USNG ||l uzpu / ;

	Hgzz	wn·li  vy
√....√.....√.....√		
	) *	)Rvz nyl USN *
USN		
		Rvz nyl USN
	)21 , *	)2 *

) 21 \*;

100만 벡터에서 가장 유사한 10개 찾기:

Faiss (C++, 최적화):
평균: 10~50ms

pgvector (PostgreSQL 위):
평균: 50~200ms

Exqutor + pgvector:
평균: 50~200ms (벡터 검색 속도는 같음)

장점: 정확한 카디널리티로 쿼리 계획 최적화  
따라서 복합 쿼리에서는 100배 향상 가능

;

벡터 검색만 필요 → Faiss  
예: "상위 1000개의 가장 유사한 벡터 찾기"

벡터 + SQL 복합 쿼리 → pgvector  
예: "의료 문헌 중 비슷한 논문을 찾되,  
특정 저자, 특정 연도, 인용도 상위 5개"

qH는그대공공본니 a도공대공본니 Q

;

/ ;

2; wn·l i |vy

ANALYZE documents; -- 벡터 통계도 수집  
  
-- 그러면 pgvector가:  
-- "이 테이블의 embedding 컬럼은 거리 분포가 X다"  
-- 자동으로 선택도 추정

-G |x~|vy /

;

wn·l i |vy ) \*

3; wn·l i |vy

향후 10년 동안 pgvector는:  
- 벡터 검색 성능만 개선  
- 통계 기반 카디널리티 추정은 미지원

이 경우, Exqutor는 계속 필요하고 가치 있음

;

G |x~|vy " " - wn·l i |vy G |x~|vy /

;

2/wn·l i |vy ) \*

3/

4/ G |x~|vy

q=: l 는그대공공본니 Q

;

;

2/NO

f wX|ovu

# TCI )TI |yP·gsC~nt l u|l k l l u| ygl p u\*

l t hl kkpun > vw ug/Gt hl kkpun/i yl gl l )

pww~|>" "-

t vkl s>"l ||.l t hl kkpun.4. sy nl "

\*

# Rvz|nyl USN, wn·l i |vy

yl z~\$ > kh/x~l yx)

UGNGEV kvi ~t l u|epk-i vu|l u| HTQO kvi ~t l u|z

```

[J GTG)l t hl kkpun =>? & z*= 1/4
CPF i y| g|l kegl ? PQ[]*. R VGTTCN(2 xl gy(
NOK/21
*
f
3/
f zxs
.. "
UGNGEV wwyk~i |epk-wugt l
HTQO wwyk~i |z w
[J GTG)w l t hl kkpun =>? & z*= 1/5
CPF wwypl = 211
CPF wylpun ? 5/1
CPF wz|vi r ? 1
f
4/
f
달 EPP 달)623 *
Rvz|nyl USN, wn·li |vy
f
5/)Cvut gxFI || i |pu*
f zxs
..
UGNGEV |ygmpep~zv~yi |epv
HTQO ul |—vyr el ygmpe
[J GTG)|ygmpe l t hl kkpun =.? uvyt gæwg|| i yu*? 1/9
CPF |p l z|gt w? PQ[]*. R VGTTCN(2 ov~y(
f
wn·li |vy ;

```

데이터 크기: 1,000만 ~ 1억 벡터  
 쿼리 패턴: 복합 SQL 쿼리 필요  
 인프라: 기존 PostgreSQL 시스템 활용 가능  
 성능 요구: 초당 100~1,000개 쿼리

wn·li |vy ;

데이터 크기: 10억개 이상 벡터  
 쿼리 패턴: 순수 벡터 검색만 필요  
 성능 요구: 초당 10,000개 이상 쿼리  
 → Milvus, Qdrant 등 전문 벡터 DB 권장

q==l a도등대니 Q

;  
 G|x~|vy )Egykpgs×Gz|p g|pu\* ;

데이터베이스 최적화의 기본 문제:  
 정확한 카디널리티 추정 → 최적 실행 계획  
 부정확한 추정 → 비최적 계획 → 성능 저하

;  
 2/Rvz|nyl USN )2: : 1 \*

3/ )3121 \*

; OUEP )O~\$pUl | Evu·vs~|pugsPl |—vyrz\*  
;

4/G |k~|vy )3131 \*  
) 211& \*

G |k~|vy ;

기준: "통계로 추정하자" (부정확함)  
Exqutor: "조건의 정확한 카디널리티를 직접 측정하자"  
→ 옵티마이저에 정확한 정보 제공  
→ 항상 최적 계획 생성

q=?! 는그대공부니 Q

;

J PU[ ;

-- HNSW는 동적 인덱스, 일반적으로 재구축 불필요  
CREATE INDEX ON documents USING hnsw (embedding vector\_l2\_ops);  
  
-- 데이터 추가/삭제 → 인덱스 자동 업데이트  
INSERT INTO documents VALUES (...); -- 인덱스도 업데이트됨  
  
-- 성능 저하 시에만 재구축  
REINDEX INDEX documents\_embedding\_idx;

K\HHsg| ;

-- IVFFlat는 정적 인덱스, 데이터 변경 후 재구축 필요  
CREATE INDEX ON documents USING ivfflat (embedding vector\_l2\_ops);  
  
-- 대량 데이터 변경 후  
DELETE FROM documents WHERE created\_at < '2024-01-01';  
  
-- 인덱스 재구축  
REINDEX INDEX documents\_embedding\_idx;

;

-- 대량 삽입 시:  
-- 1. 인덱스 비활성화  
ALTER INDEX documents\_embedding\_idx UNUSABLE;  
  
-- 2. 데이터 삽입  
INSERT INTO documents (...); -- 매우 빠름  
  
-- 3. 인덱스 재구축  
REINDEX INDEX documents\_embedding\_idx;

C8 2C 3

C8= 2A: 3

- G |k~|vy; G ||l ukl k S~l yx Qw|p p† ymxyYl i |vy.g~nt l u|l k Cugx|p gsS~l yϕ z /  
Rvz|nyl USN 2-111 /  
/  
/

C8 2= 3

/

wn·li|vy ;

QW uCKCu|oyw~l vvns /

wn·li|vy Rvz|nyl USN - 23-111 I pJ~h /

wn·li|vy "Rvz|nyl USN " / USN 겠 - - 겠

/

;

wn·li|vy / 44/4& /

USN & - Rvz|nyl USN /

C8A 2= 3

/

;

211 - 879 / " " ;

```
SELECT * FROM papers
WHERE (embedding <=> query_embedding) < 0.2;
```

Rvz|nyl USN ;

; 44/4& ) \*

; 211 7|1/444 > 444-444

;" - "

; Ul x~l u|psUi gu

;

; 1/2& ) \*

; 211 7|1/112 > 2-111

;" 44/4& 1/2& A "

;

Ul x~l u|psUi gu; 6 ) 211 \*

J PU[ ) \*; 61t z)J PU[ 2-111 \*

; 211

/

C8B 2A: 3

;

2/"wn·li|vy 44/4& "

; wn·li|vy -

; wn·li|vy

3/" FD "

; Rvz|nyl USN - -

;

4/" "

;

Rvz|nyl USN /

C8a 등대본 2= 3

```

;

Rvz|nyl USN "wsguul y ovvr"
/

;

```

1. 벡터 조건 감지  
(embedding <=> query) < 0.2를 발견
2. 정확한 카디널리티 직접 측정  
HNSW 인덱스에서 실제로 범위 검색을 수행 (1~2ms)
3. PostgreSQL에 정보 제공  
"이 조건의 정확한 선택도는 0.1%입니다"
4. 최적 실행 계획 수립  
정확한 카디널리티로 다시 최적화

```

;

기존 PostgreSQL: 333,333개 예상, Sequential Scan, 5초
Exqutor: 1,000개 예상, Index Scan, 50ms
성능 향상: 100배

```

```

;

wn·li|vy
Rvz|nyl USN
3t z

```

C8D 2= 3

```

/

;

Rvz|nyl USN25, wn·li|vy1/6
JPU[)t >27-l mē vuz|y~i|p̄u>311*
4)879-2647 *
2 ² 2
;

```

		)t z*	G k~ vy)t z*	
√.....√.....√.....√.....√.....√				
		3-261	56	59
		6-541	: 6	68
		29-311	46	631
,		53-611	63	2-12:

- /

C8E a도대답니 2A: 3

```

;

1. Extension 설치
CREATE EXTENSION exqutor;

2. 활성화
SELECT exqutor_enable();

3. 이후 모든 쿼리가 자동으로 최적화됨
SELECT * FROM documents WHERE (embedding <=> query) < 0.3;
-- 자동으로 정확한 카디널리티 사용

```

C8 2: 3

```

G ||x~|vy ;
2/) *
3/J PU[
4/ 2^2 3t z
;
wn·li |vy -G ||x~|vy / ;
wn·li |vy
G ||x~|vy
Rvz|nyl USN

```

C8H 2: 3

;

pgvector + PostgreSQL은 강력한 조합입니다.  
 하지만 부정확한 카디널리티 추정이 성능을 심각하게 저하시킵니다.

Exqtur는 이 문제를 간단하고 효과적으로 해결합니다:

- PostgreSQL 공식 메커니즘만 사용
- 기존 코드 수정 불필요
- 최대 1,000배 성능 향상

```

;
Rvz|nyl USN
AG ||x~|vy /
/

```

## D8

### D8는 대공방호니

```

pgvector 설치
git clone https://github.com/pgvector/pgvector.git
cd pgvector
make
sudo make install

PostgreSQL에서 활성화
psql -U postgres -d mydb
CREATE EXTENSION vector;

```

### D8

```

-- HNSW 인덱스 생성
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops)
WITH (m=16, ef_construction=200);

-- 쿼리 시 ef 파라미터 조절
SET hnsw.ef = 32;

-- VACUUM으로 인덱스 정리
VACUUM ANALYZE documents;

```

### D8A

```
2; "
```

ERROR: vector dimensions mismatch

원인: 다른 크기의 벡터 비교

해결: 벡터 생성 모델 통일 (모두 768차원 등)

```
3;
```

쿼리가 Sequential Scan을 선택하는 경우

원인: 선택도 추정이 높게 나와서 (33.3%)

해결: Exqutor 사용, 또는 강제로 Index Scan 권장 (HINT 사용)

DB a능대본니

```
2/ ;wn·l i|vy 44/4&
3/ ;Rvz|nyl USN
4/ ; 2-111
5/ ;
```

```
G|x~|vy - ;
wn·l i|vy) - -JPU[-KHHg]*
44/4&
G|x~|vy)wguul yovvr *
23
6
2-811,
;
;
; 5 S' C
; 6 6
; 2^2 4
```